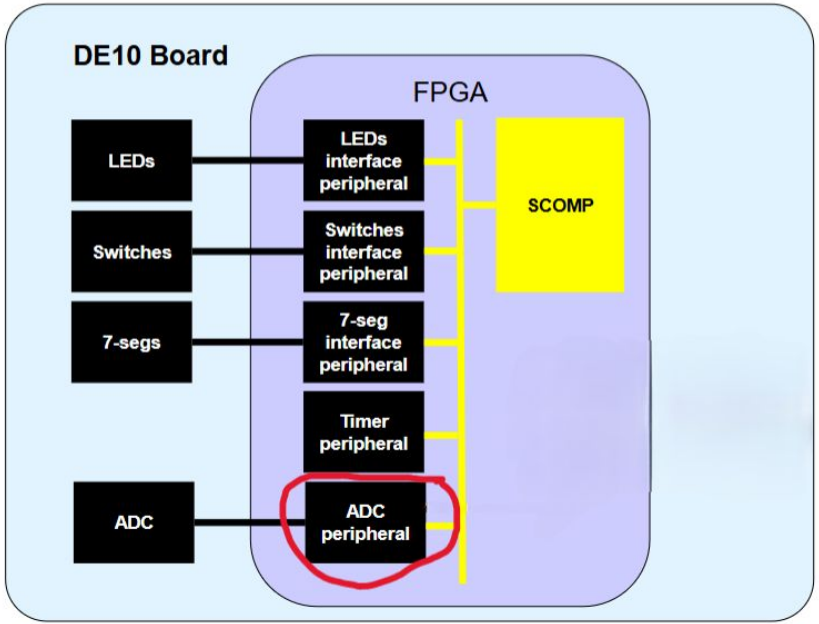
LTC2308 Peripheral

Andrew, Wei Chen, Vendang, Jinyoung, Arjun

Design Problem

- There is no way to interact with the ADC through SCOMP.
- Solution:
  - Create a peripheral that would allow SCOMP programmers to be able to interact with the ADC.
  - The peripheral would:
    - Use a fixed single channel configuration.
    - Continuously sample and store the latest sampled data.



Technical Approach

Design:

- Two read-only SCOMP addresses: 0xC0 for ADC data and 0xC1 for ADC status.
- Keeps the interface simple by using a fixed single-channel configuration and continuous sampling.
- Returns the most recent 12-bit sampled value at 0xC0, while 0xC1[1] provides a “busy” indicator.

Operation:

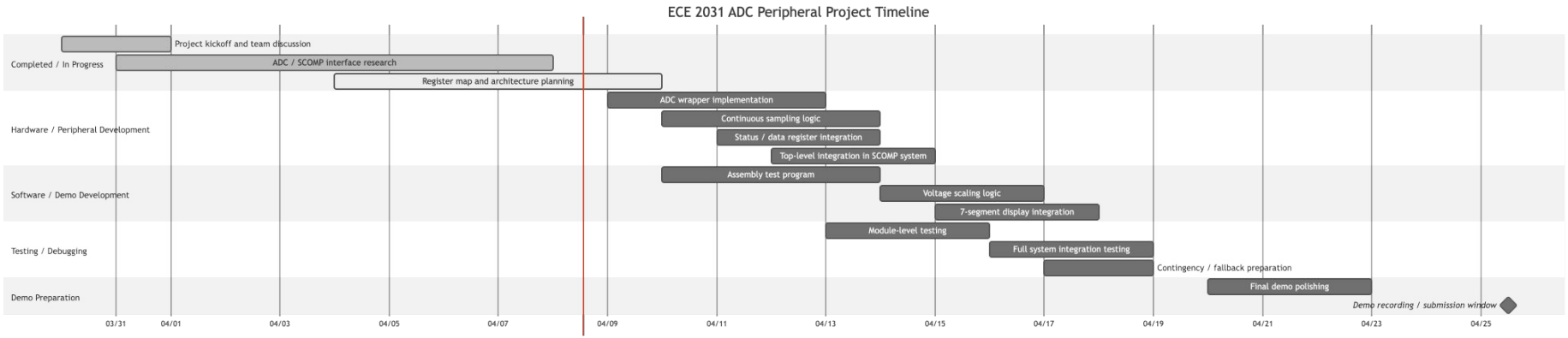
- After a reset, the peripheral automatically starts a conversion.
- When a conversion completes, the new sampled value is stored and the next conversion will begin immediately.
- SCOMP software reads 0xC0 at any time for the latest value and may read 0xC1 to check whether an update is in progress.

Technical Approach: Demonstration

- What will we use?
  - Potentiometers on the protoboard
  - ADC
  - Seven-segement display
- What are we making?
  - We plan on making a voltmeter, a device that will show the instantaneous voltage that is on the input of the ADC.

Header pin receives Voltage → ADC converts Voltage to a Number → Number ∝ Voltage

Management Plan



Phase 1
- Study SCOMP I/O interface
- Analyze ADC controller
- Design register map and architecture

Phase 2
- Implement ADC wrapper
- Add continuous sampling logic
- Integrate with SCOMP system

Phase 3
- Develop SCOMP test program
- Convert ADC values to voltage
- Display results on HEX display

Phase 4
- Verify module functionality
- Test full system integration
- Prepare fallback demo options

Phase 5
- Final system validation
- Prepare demonstration workflow

Division of Labor

Vedang
- Integrate LTC2308 controller with SCOMP system
- Verify SPI timing and signal behavior
- Implement continuous sampling control logic

Arjun
- Design memory-mapped register interface (0xC0 / 0xC1)
- Implement IO_ADDR decoding logic
- Coordinate module integration across team

Andrew
- Develop assembly programs for ADC data access
- Implement voltage scaling logic
- Support peripheral interface testing

Jinyoung
- Connect ADC output to HEX display
- Implement real-time visualization pipeline
- Prepare demonstration workflow

Wei Chen
- Perform simulation-based module testing
- Validate ADC data correctness across reads
- Support system-level debugging during integration

Contingency plan

- **Risk 1 — Continuous sampling instability**
  - Fallback: switch to software-triggered conversion mode
  - Manual OUT instruction initiates sampling
- **Risk 2 — Voltage scaling implementation delay**
  - Fallback: display raw 12-bit ADC output directly
  - Still demonstrates correct peripheral functionality
- **Risk 3 — HEX display integration failure**
  - Fallback: redirect output to LED register
  - Maintains real-time visualization capability
- **Risk 4 — SPI timing mismatch with ADC**
  - Fallback: adjust controller clock divider
  - Ensures stable communication with LTC2308
- **Risk 5 — Register interface conflicts during integration**
  - Fallback: simplify interface to single read-only data register (0xC0)

Conclusion

- We're building an ADC peripheral that gives SCOMP seamless read access to analog voltage data via two I/O addresses. `0xC0` for the 12-bit sample and `0xC1` for conversion status.
- The peripheral runs continuously with no manual triggering required.
- We'll demonstrate it end-to-end: a potentiometer feeds a voltage into the ADC, the peripheral samples it, and an SCOMP program multiplies the raw value by a constant, and displays the voltage on the seven-segment display.
- Our team is split across their majors: Vedang (EE) will be leading the charge on the hardware and Arjun (CS) will be leading the software side due to previous experience with Agile methodology.